

Лабораторная работа №1. Вычисление числа π с использованием OpenMP

Марданов Самир, группа 11-401

Описание работы. Реализованы последовательная и параллельная версии вычисления π по ряду Лейбница $\pi = 4 \sum_{i=0}^{n-1} (-1)^i / (2i + 1)$; параллельная версия использует `#pragma omp parallel for reduction(+:sum)`, время измерялось через `omp_get_wtime()`. Программа собрана `gcc -O2 -fopenmp` (GCC 11.4) и запускалась на 4 ядрах Apple Silicon (Linux, aarch64). Для каждой конфигурации взята медиана трёх запусков (для $n = 10^{10}$ — один запуск).

Эксперимент 1. $n = 10^9$			Эксперимент 2. 8 потоков			
Потоки	Время, с	Ускорение	n	Послед., с	OpenMP (8), с	Ускорение
1	0.973	0.99	10^7	0.0104	0.0040	2.60
2	0.504	1.92	10^8	0.0965	0.0271	3.56
4	0.256	3.78	10^9	0.964	0.260	3.71
8	0.279	3.47	10^{10}	9.621	2.525	3.81
16	0.259	3.73	Абс. ошибка при $n = 10^{10}$: $\approx 1.0 \cdot 10^{-10}$			
Последовательно: 0.967 с						

Вывод. При увеличении числа потоков с 1 до 4 время выполнения сократилось почти пропорционально: с 0.97 до 0.26 с, ускорение 3.78 при теоретическом максимуме 4. Это объясняется тем, что члены ряда независимы, а единственная точка синхронизации — финальная редукция суммы. Начиная с 4 потоков ускорение перестало расти: на 8 и 16 потоках оно осталось на уровне 3.5–3.7, так как машина имеет 4 вычислительных ядра и лишние потоки лишь конкурируют за них, добавляя накладные расходы на переключение (отсюда небольшой провал на 8 потоках). Однопоточная OpenMP-версия работает так же, как последовательная, то есть накладные расходы самого OpenMP пренебрежимо малы. Размер задачи существенно влияет на эффективность: при $n = 10^7$ вычисления занимают около 10 мс, и заметную долю времени съедает создание потоков и распределение итераций — ускорение лишь 2.6. С ростом n полезная работа растёт линейно, а накладные расходы остаются постоянными, поэтому ускорение приближается к числу ядер (3.81 при $n = 10^{10}$). Таким образом, OpenMP эффективен для достаточно больших задач, а число потоков разумно выбирать равным числу физических ядер.